

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО»

ПРОГРАМУВАННЯ ВЕБЗАСТОСУНКІВ

Лекція 2

Рекомендовано Методичною радою КПІ ім. Ігоря Сікорського
як навчальний посібник для здобувачів ступеня бакалавра
за освітньою програмою «Інженерія програмного забезпечення інтелектуальних кібер-
фізичних систем в енергетиці»
за спеціальністю 121 Інженерія програмного забезпечення

Електронне мережеве навчальне видання

Київ
КПІ ім. Ігоря Сікорського
2024

ПЛАН ЛЕКЦІЇ

1.2. ПЕРША ПРОГРАМА	3
1.2.1. Створення програми.....	3
1.2.2. Компіляція та виконання програми	4
1.3. Go в LITEIDE	6
1.4. Go у VISUAL STUDIO CODE	11
РЕЗЮМЕ	14
КОНТРОЛЬНІ ЗАПИТАННЯ І ЗАВДАННЯ.....	15
ОГЛЯД ТЕСТОВИХ ЗАВДАНЬ	16
<i>Закриті запитання з одним варіантом відповіді</i>	<i>16</i>
Що означає директива package main у програмі Go?	16
<i>Закриті запитання з кількома правильними відповідями.....</i>	<i>16</i>
Які середовища підтримують розробку на Go?	16
<i>Завдання на відповідність.....</i>	<i>16</i>
Встановіть відповідність між командою та її функцією:	16
<i>Завдання на доповнення коду.....</i>	<i>17</i>
Доповніть програму для виведення "Hello, Go!":.....	17
<i>Завдання з вибором істинності.....</i>	<i>17</i>
Команда go build створює виконуваний файл програми.....	17
<i>Відкриті запитання.....</i>	<i>17</i>
Поясніть, чому команда go run не створює виконуваного файлу.....	17
<i>Завдання з кодом</i>	<i>17</i>
Яка помилка в програмі?.....	17
<i>Завдання на завершення коду.....</i>	<i>18</i>
Завершіть код, щоб вивести:.....	18
ПЕРЕЛІК ПОСИЛАНЬ.....	19
СПИСОК РЕКОМЕНДОВАНОЇ ЛІТЕРАТУРИ І ЕЛЕКТРОННИХ	
РЕСУРСІВ.....	21

1.2. Перша програма

Створимо першу програму мовою Go. Для написання коду нам знадобиться текстовий редактор. Можна взяти будь-який редактор, наприклад, вбудований блокнот чи популярний Notepad++ чи будь-який інший. Для трансляції вихідного коду у застосунок потрібен компілятор.

1.2.1. Створення програми

Визначимо на жорсткому диску теку для зберігання файлів із вихідним кодом. Нехай це буде тека **C:\golang**. У цій теці створимо новий текстовий файл, який назвемо **hello.go**, як показано на рисунку 1.12.

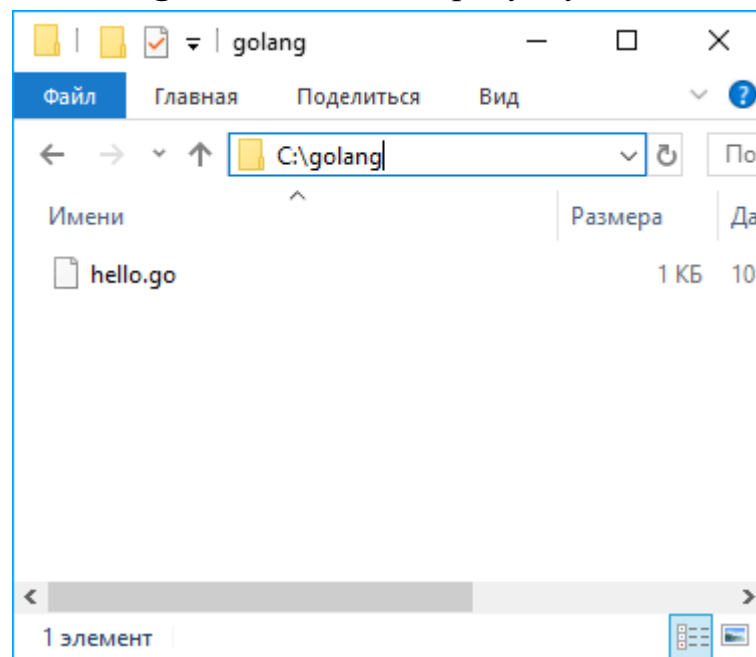


Рисунок 1.12. Вікно створення нового текстового файлу

Відкриємо цей файл у будь-якому текстовому редакторі та визначимо в ньому наступний код:

```
1 package main
2 import "fmt"
3
4 func main() {
5     fmt.Println("Hello Go!")
6 }
```

Програма мовою Go визначається як набір пакетів. Програмний код повинен бути визначений у певному пакеті. Тому на початку файлу за

допомогою оператора `package` вказується, до якого пакету належатиме файл. В наведеному прикладі це пакет **main**:

```
1 package main
```

Причому пакет повинен називатися саме **main**, оскільки саме цей пакет визначає файл, що виконується.

При складанні програмного коду може знадобитися функціонал інших пакетів. У мові Go є багато вбудованих пакетів, які містять код, що виконує певні дії. Наприклад, у наведеному прикладі виводиться повідомлення на консоль. І для цього було використано функцію **Println**, яка визначена в пакеті **fmt**. Тому у другому рядку за допомогою директиви **import** було підключено сам цей пакет наступним чином:

```
1 import "fmt"
```

Далі йде записано функцію **main**. Це головна функція будь-якої програми на мові Go. Насправді все, що виконується у програмі, виконується саме функцією **main**.

Визначення функції починається зі слова **func**, після якого вказується назва функції, тобто **main**. Після назви функції у дужках вказується перелік параметрів. Оскільки функція **main** не приймає жодних параметрів, то в даному випадку вказуються порожні дужки.

Потім у фігурних дужках визначається тіло функції **main**, тобто ті дії, які і виконує функція:

```
1 func main() {
```

В наведеному прикладі функція **main** виводить на консоль рядок **"Hello Go!"**. Для цього застосовується функція **Println()**, яка визначена у пакеті **fmt**. Тому при виклику функції спочатку вказується ім'я пакета та через крапку ім'я функції. В дужках функції передається повідомлення, яке вона повинна вивести на консоль, наприклад:

```
1     fmt.Println("Hello Go!")
```

1.2.2. Компіляція та виконання програми

Тепер скомпілюємо та виконаємо цю програму. Для цього необхідно передати файл із вихідним кодом компілятору **go.exe** та вказати потрібну команду. Для цього необхідно відкрити командний рядок, або термінал, і перейти в ньому за допомогою команди **cd** до теки, де зберігається файл з вихідним кодом **hello.go**, наприклад це тека **C:\golang**, як показано на рисунку 1.13.

```
cd C:\golang
```

Рисунок 1.13. Зовнішній вигляд вікна введення команди

Потім запустимо програму на виконання за допомогою команди, як показано на рисунку 1.14.

```
go run hello.go
```

Рисунок 1.14. Зовнішній вигляд вікна введення команди

В наведеній конструкції **go** позначає компілятор. Оскільки при установці шлях до компілятора автоматично прописується в змінній **PATH** у змінних оточення, то не потрібно вказувати повний шлях **C: Go bin go.exe**, а достатньо вказати просто ім'я програми **go**. Далі записується параметр **run**, який вказує на необхідність виконання програми. І насамкінець вказується власне файл програми **hello.go**.

У результаті після виконання програми на консоль буде виведено повідомлення " **Hello Go!** ", як показано на рисунку 1.15.

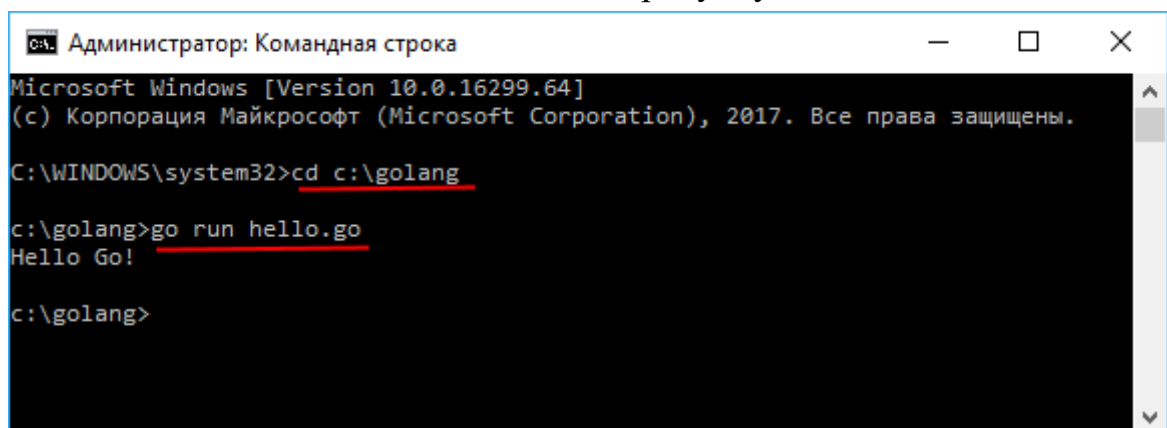


Рисунок 1.15. Результат роботи програми

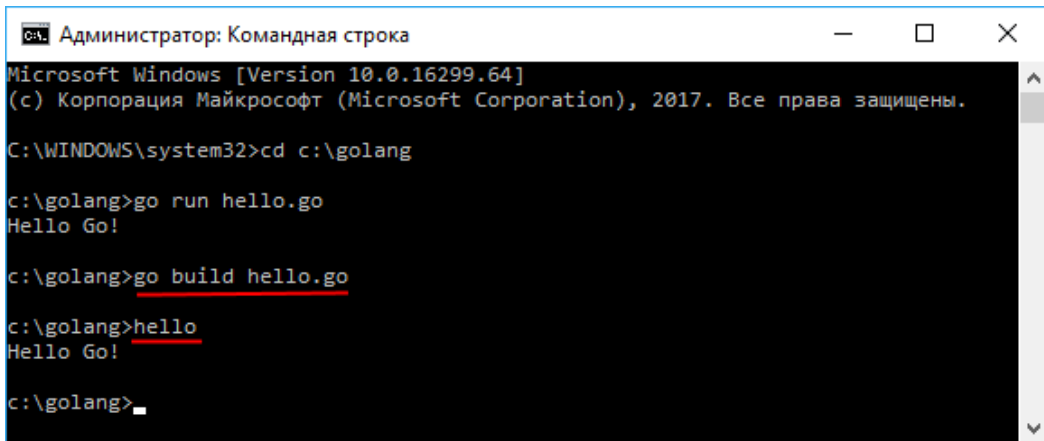
Наведена команда запускає на виконання програму, але не компілює програму в окремий файл. Для компіляції необхідно виконати іншу команду, як показано на рисунку 1.16.

```
go build hello.go
```

Рисунок 1.16. Зовнішній вигляд вікна введення команди

Після виконання цієї команди у теці з вихідним файлом з'явиться ще один файл, який називатиметься **hello.exe** і який вже можна запускати, як звичайний

виконуваний файл. Після цього можна виконати програму, запустивши в консолі цей файл, як показано на рисунку 1.17.



```
Администратор: Командная строка
Microsoft Windows [Version 10.0.16299.64]
(c) Корпорация Майкрософт (Microsoft Corporation), 2017. Все права защищены.

C:\WINDOWS\system32>cd c:\golang

c:\golang>go run hello.go
Hello Go!

c:\golang>go build hello.go

c:\golang>hello
Hello Go!

c:\golang>
```

Рисунок 1.17. Результат роботи програми

1.3. Go в LiteIDE

Використання інтегрованих середовищ розробки (IDE) часом спрощує управління проєктом і створення програми. Для мови Go одним з найпопулярніших середовищ розробки є LiteIDE. Це безкоштовне кросплатформне середовище, яке можна вільно завантажити на робочий комп'ютер. Офіційний сайт IDE LiteIDE можна знайти за посиланням <https://code.google.com/archive/p/liteide/>.

Безпосередньо завантажити всі файли цієї IDE можна за посиланням <https://github.com/visualfc/liteide>, як показано на рисунку 1.18.

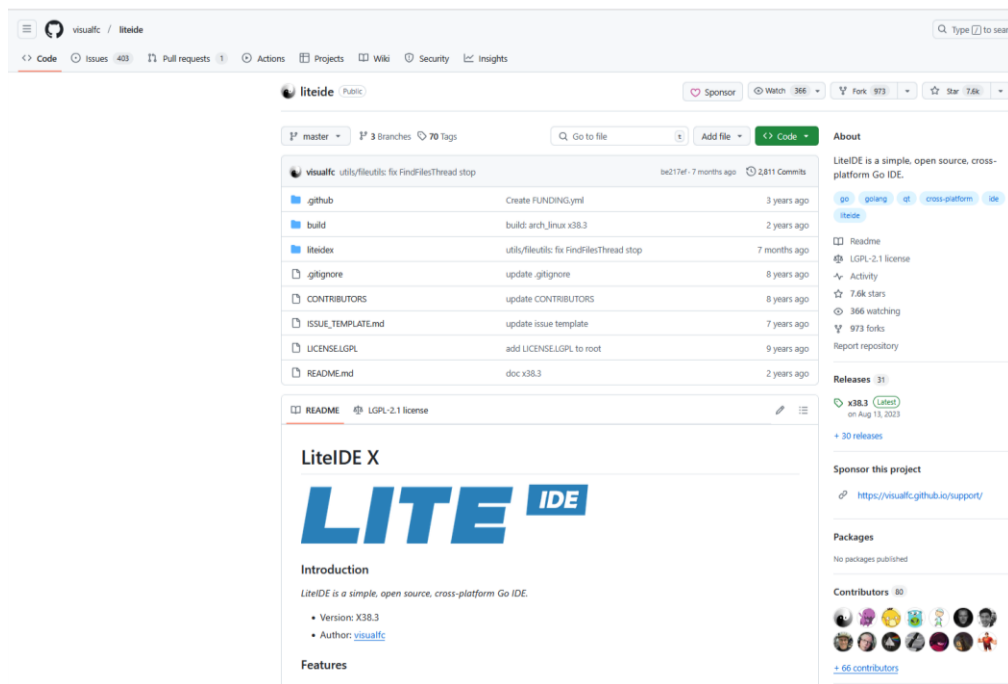


Рисунок 1.18. Зовнішній вигляд офіційної сторінки завантаження пакету LiteIDE

На цій сторінці можна знайти посилання для завантаження. Причому нічого не потрібно встановлювати. За посиланням завантажується zip-архів, який після завантаження достатньо розпакувати. Після розпакування архіву в кореневій теці в каталозі **bin** можна знайти файл виконуваної програми **liteide.exe** (на Windows), через який можна запустити середовище розробки, як показано на рисунку 1.19.

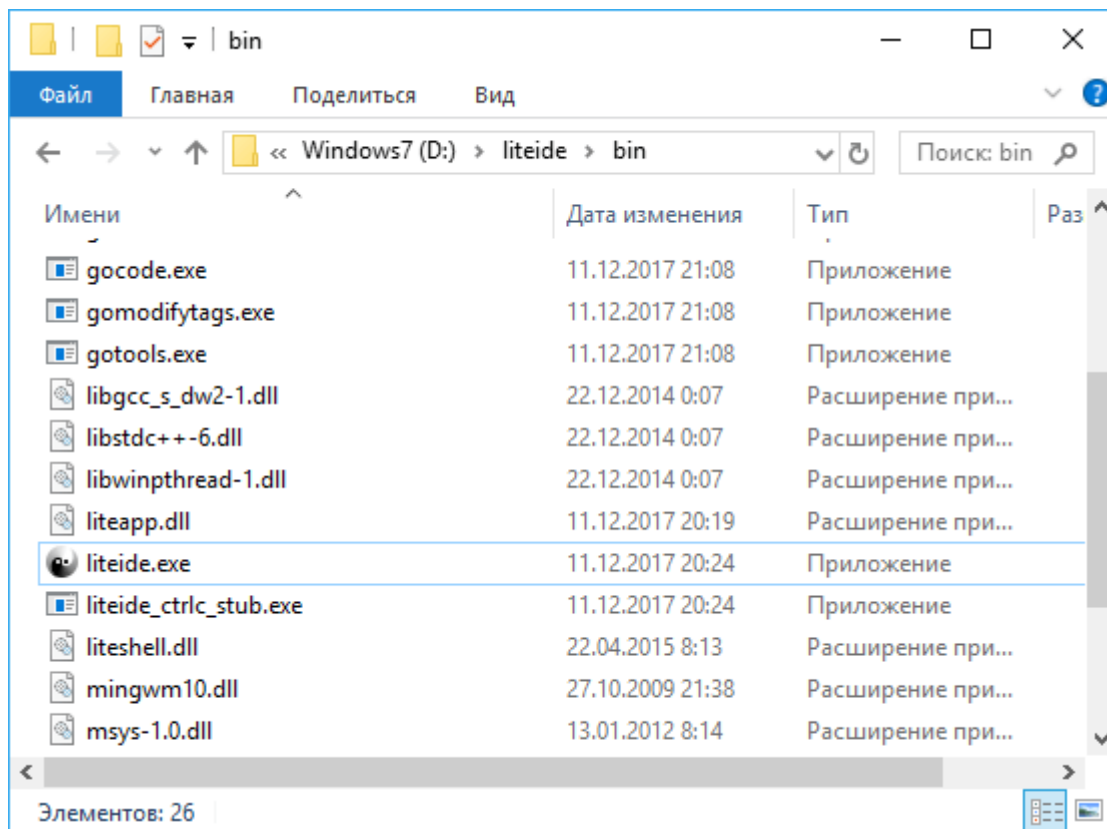


Рисунок 1.19. Склад завантаженого пакету LiteIDE

Запустимо цей файл, і за замовчуванням відкриється стартова сторінка. LiteIDE має локалізовані версії, і через параметри можна переключати мову середовища на будь-яку із запропонованих.

Для створення нового проєкту перейдемо до меню **File(Файл) -> New (Створити)**, як показано на рисунку 1.20.

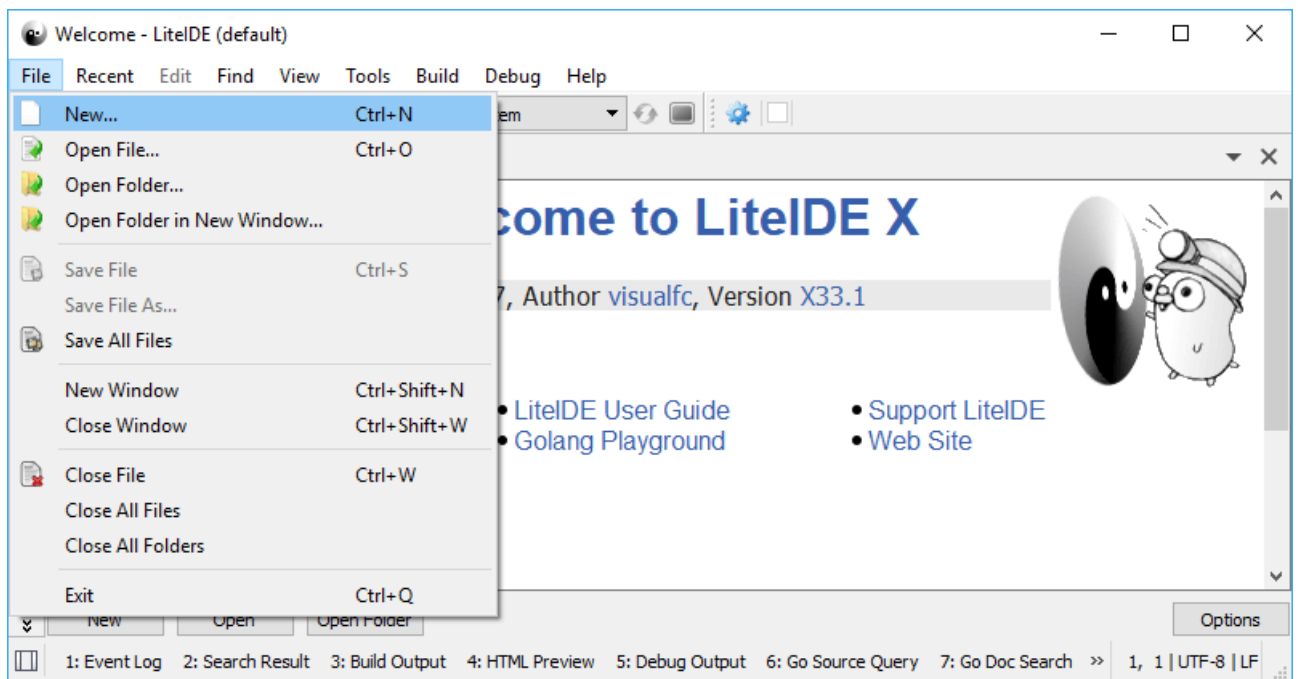


Рисунок 1.20. Вікно створення нового проєкту

Після цього з'явиться вікно вибору шаблону проєкту, як показано на рисунку 1.21.

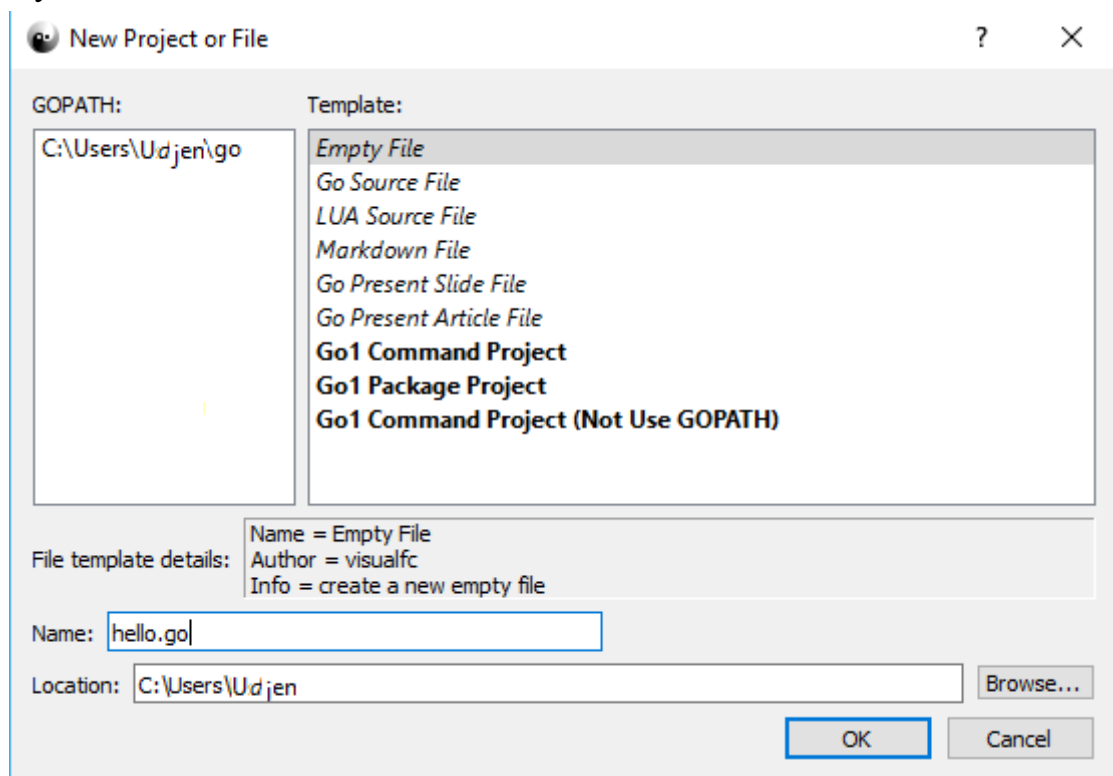


Рисунок 1.21. Вікно вибору шаблону проєкту

Як шаблон виберемо **Empty File**. А знизу в полі **Name** вкажемо для файлу ім'я **hello.go**. В іншому полі можна побачити шлях до теки, де розташовуватиметься файл. Цей шлях також можна змінити. Після цього буде

створено проєкт і відкриється вікно, де необхідно буде підтвердити відкриття проєкту, як показано на рисунку 1.22.

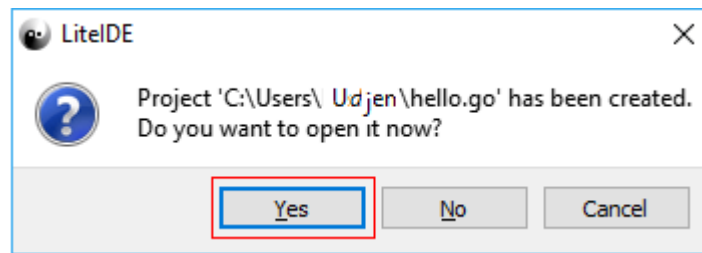


Рисунок 1.22. Вікно підтвердження відкриття проєкту

Потім у центральній частині вікна програми відкриється текстовий редактор, як показано на рисунку 1.23. Введемо до нього наступний код:

```
1  package main
2
3  import "fmt"
4
5  func main() {
6      fmt.Println("Hello Go")
7  }
```

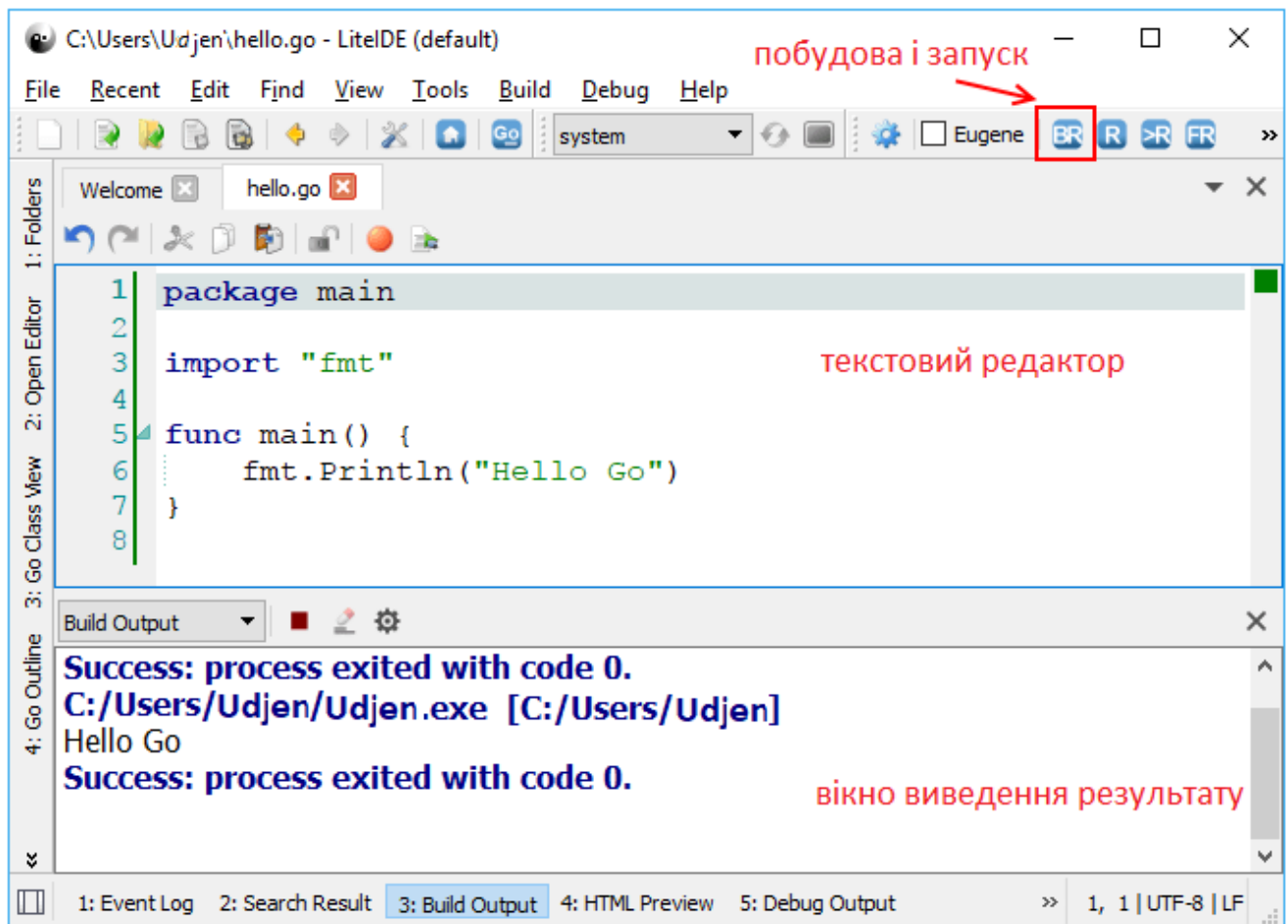


Рисунок 1.23. Зовнішній вигляд LiteIDE

Для збірки та запуску проєкту необхідно натиснути на панелі інструментів кнопку **BR**. І внизу вікна IDE з'явиться поле виведення, де можна побачити результат виконання програми.

Також у LiteIDE можна створювати та використовувати інші типи проєктів. Наприклад, створимо проєкт **Go1 Command Project**, який нехай називається **hello**, як показано на рисунку 1.24.

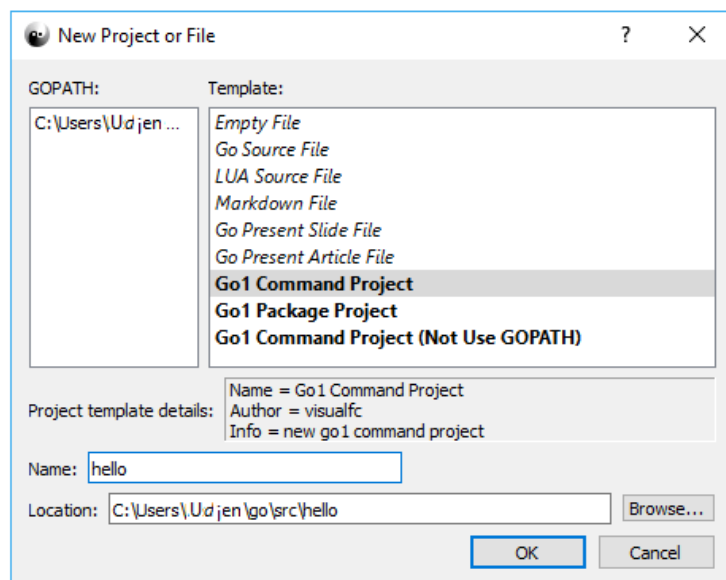


Рисунок 1.24. Створення проєкту Go1 Command Project

В результаті буде створено проєкт, який розташовуватиметься в окремій теці та який за замовчуванням складатиметься з двох файлів, як показано на рисунку 1.25.

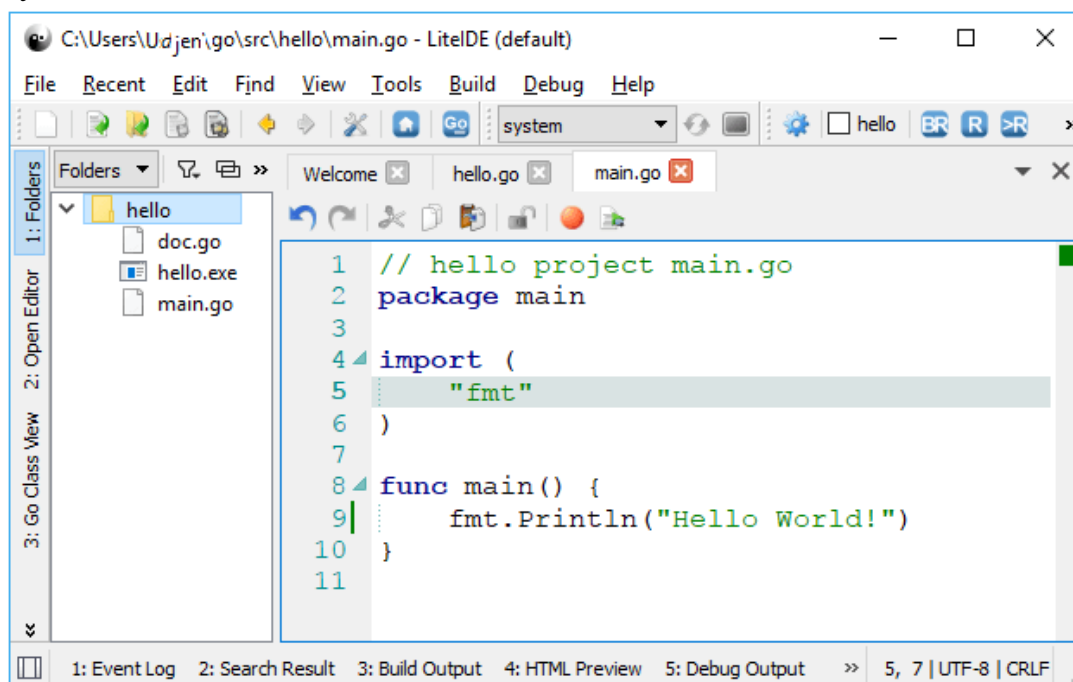


Рисунок 1.25. Зовнішній вигляд LiteIDE для проєкту Go1 Command Project

1.4. Go у Visual Studio Code

Інтегроване середовище розробки Visual Studio Code представляє кросплатформний потужний та легковажний текстовий редактор від компанії Microsoft, який підтримує підсвічування синтаксису, інтелектуальну підказку для різних мов програмування та багато іншого. Розглянемо, як можна використовувати цей текстовий редактор для розробки мовою Go.

Насамперед необхідно встановити цей текстовий редактор. Інсталятор для потрібної операційної системи (є підтримка для Windows, Mac OS, Linux) можна знайти за посиланням <https://code.visualstudio.com/>.

Після інсталяції Visual Studio Code за замовчуванням немає жодної підтримки мови Go. Тому необхідно встановити відповідне розширення. Для цього необхідно перейти до **Visual Studio Code** в меню **View -> Extensions (Розширення)**, як показано на рисунку 1.26.

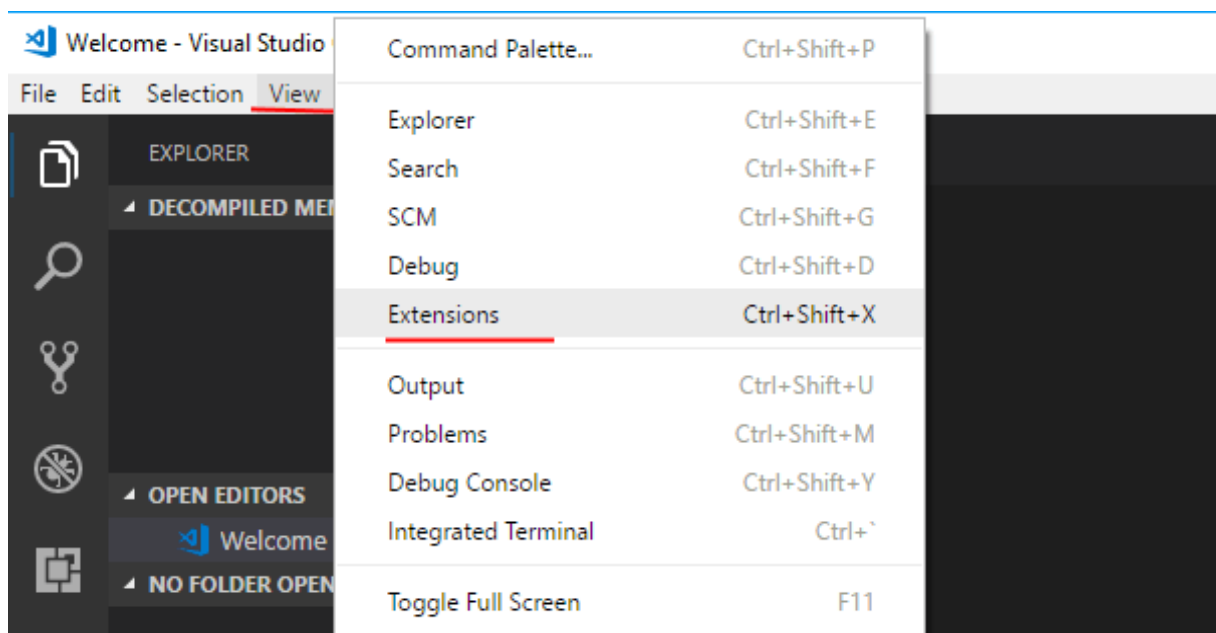


Рисунок 1.26. Вікно встановлення розширень для Visual Studio Code

У рядок пошуку розширень необхідно ввести **"go"**, після чого з'явиться список знайдених розширень. Потрібно інсталиувати перше розширення у цьому списку, яке має найбільше зкачувань, як показано на рисунку 1.27.

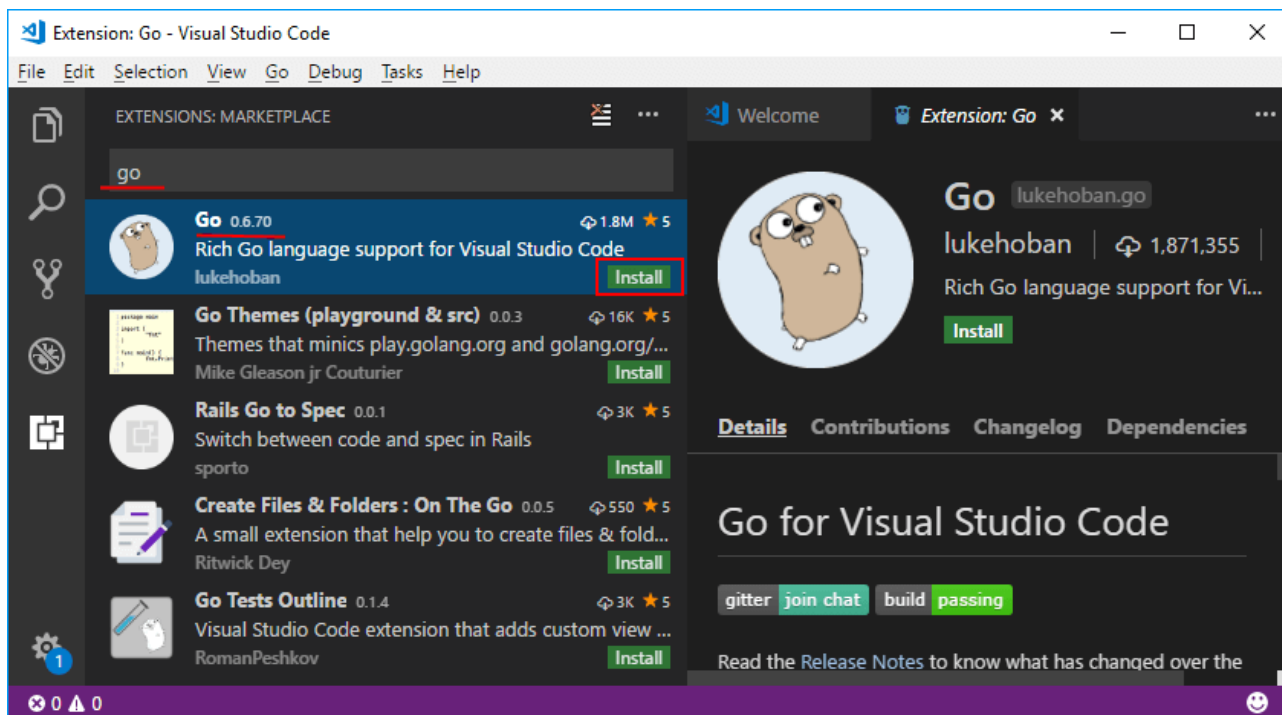


Рисунок 1.27. Вікно пошуку та вибору розширення Go для Visual Studio Code

Після інсталяції розширення необхідно визначити на жорсткому диску теку для зберігання файлів проєкту та відкрити цю папку у Visual Studio Code. Відкрити теку можна через пункт меню **File->Open Folder**. Потім створимо у **Visual Studio Code** новий файл, який назвемо **main.go**, як показано на рисунку 1.28.

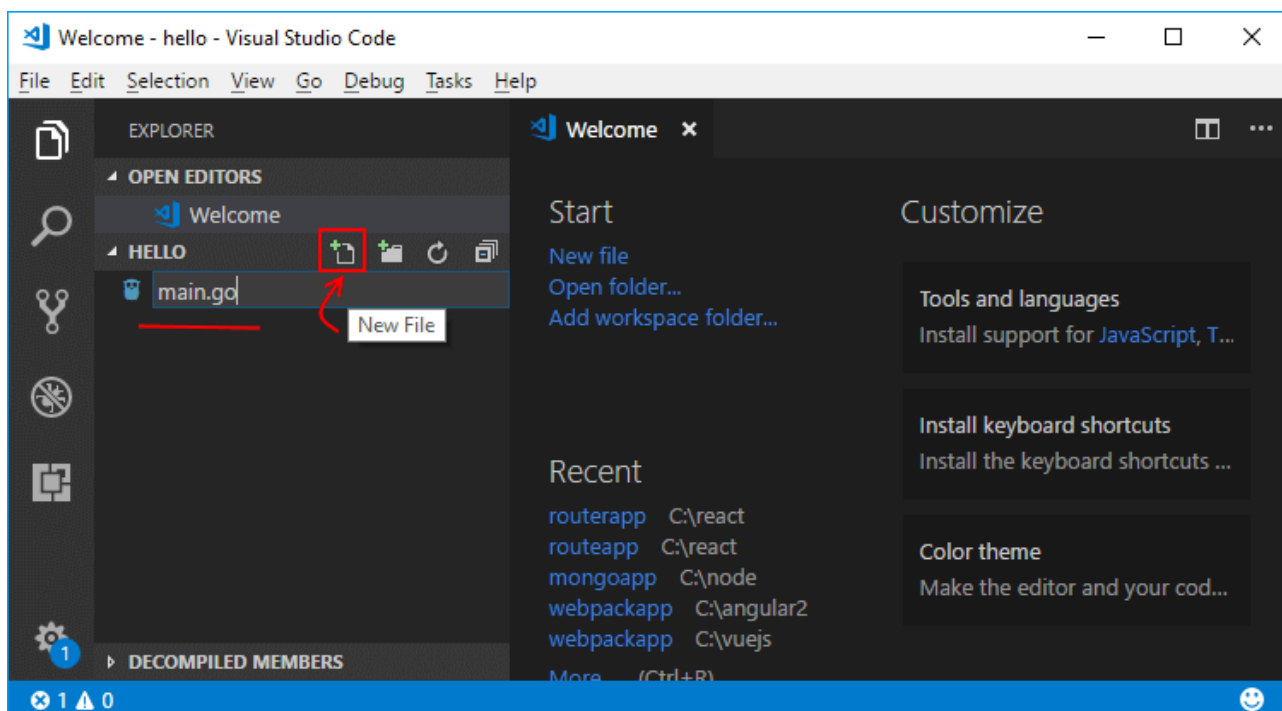


Рисунок 1.28. Вікно створення нового файлу

Відкриємо файл **main.go** і введемо до нього наступний код:

```

1  package main
2
3  import "fmt"
4
5  func main() {
6
7      fmt.Println("Hello Go")
8  }

```

Збережемо введений код, натиснувши комбінацію **Ctrl+S**, як показано на рисунку 1.29.

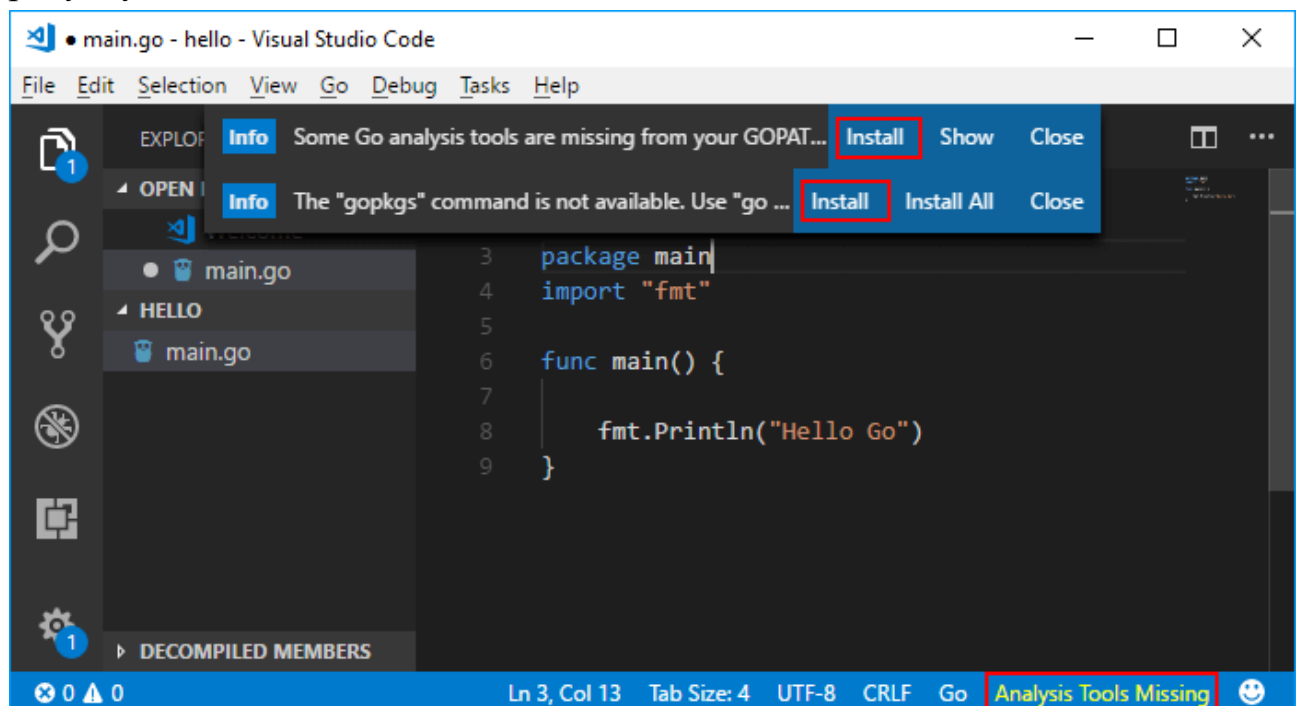


Рисунок 1.29. Вікно збереження програмного коду у новому файлі

Під час роботи з файлами **go** у **Visual Studio Code** можуть з'являтися різні повідомлення про необхідність встановлення додаткових плагінів. Наприклад, внизу вікна у статусному рядку може відображатись повідомлення "**Analysis Tools Missing**". Можна натиснути на це повідомлення, і нагорі **Visual Studio Code** відобразиться список пакетів, які бажано інсталиювати додатково. Для їх встановлення необхідно натиснути кнопку **Install**.

Перевагою **Visual Studio Code** є те, що редактор має вбудований термінал. Відкривається термінал через пункт меню **View -> Integrated Terminal**. Після цього у вікні **Visual Studio Code** знизу відкриється вбудований термінал. За замовчуванням відкривається поточна тека проєкту.

Введемо в термінал команду, як показано на рисунку 1.30, та натиснемо **Enter**.

go run main.go

Рисунок 1.30. Зовнішній вигляд вікна введення команди

Після цього у вбудованому терміналі відобразиться результат роботи програми, як показано на рисунку 1.31.

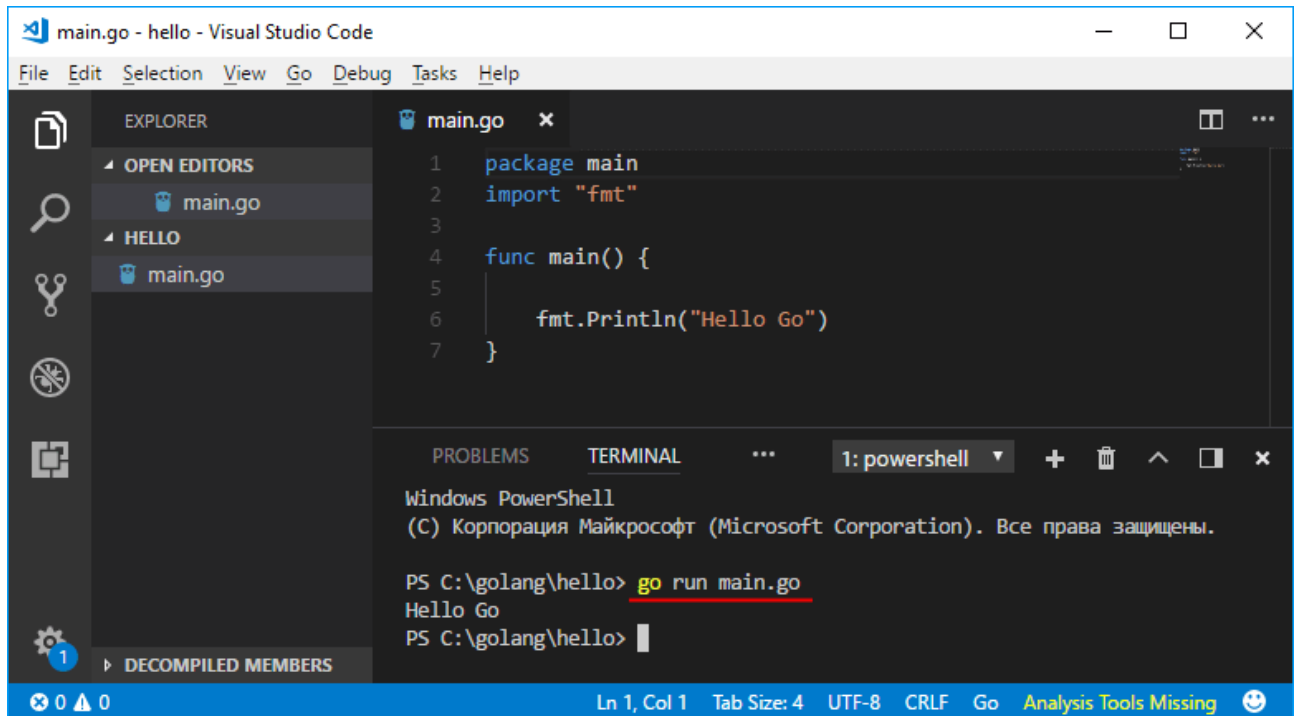


Рисунок 1.31. Результат роботи програми

Резюме

Викладений матеріал демонструє базові аспекти розробки мовою програмування Go. Описано процес створення та виконання першої програми, включаючи налаштування середовища розробки. Пояснюється структура програми, імпорт бібліотек, визначення функції main та її роль у виконанні. Наведені приклади використання команд для компіляції та запуску програм у консолі. Також розглянуто використання інтегрованих середовищ розробки, таких як LiteIDE та Visual Studio Code, із детальними інструкціями щодо встановлення, налаштування та роботи з ними. Матеріал забезпечує зрозумілий вступ до розробки на Go

Наступні розділи дозволять отримати більш глибокі знання та навички з мережевого програмування, інтеграції вебзастосунків з базами даних та створення динамічних вебзастосунків на прикладі повноцінного вебсайту.

Контрольні запитання і завдання

1. Яка основна роль функції **main** у програмі мовою Go?
2. Що означає директива **package main**?
3. Як підключити бібліотеку **fmt** у програмі Go?
4. Яка команда використовується для виконання програми Go без створення окремого виконуваного файлу?
5. Як створити виконуваний файл програми Go?
6. Яке значення має команда **go build hello.go**?
7. Чому ім'я пакета у програмі, що виконується, має бути саме **main**?
8. Що відбудеться після виконання команди **go run hello.go**?
9. Як виглядає базовий шаблон програми на Go?
10. Для чого використовується середовище LiteIDE?
11. Де можна завантажити LiteIDE?
12. Як у LiteIDE створити новий файл для програми Go?
13. Які переваги має використання Visual Studio Code для розробки на Go?
14. Як у Visual Studio Code встановити підтримку мови Go?
15. Як запустити програму Go у вбудованому терміналі Visual Studio Code?
16. Яка основна функція команди **import** у Go?
17. Чи обов'язково використовувати параметри у функції **main**? Чому?
18. Що означає метод **Println** у бібліотеці **fmt**?
19. Як визначити теку для збереження вихідного коду програми Go?
20. Які основні кроки слід виконати для запуску програми Go в командному рядку?
21. Напишіть програму мовою Go, яка виводить на екран ваше ім'я та улюблене хобі.
22. Додайте у програму імпорт бібліотеки **time** та виведіть поточний час за допомогою функції з цієї бібліотеки.
23. Створіть файл **hello.go**, напишіть у ньому програму, яка виводить «Hello, Go!». Скомпілюйте файл у виконуваний файл **hello.exe** і запустіть його.
24. Створіть програму з двома функціями: одна виводить повідомлення «Привіт!», а інша — «До побачення!». Викличте ці функції у потрібному порядку з функції **main**.
25. Напишіть програму, яка додає два числа і виводить результат. Передайте числа у функцію **main** як змінні.

26. Встановіть LiteIDE, створіть новий проєкт, напишіть програму, яка обчислює квадрат числа, та запустіть її через LiteIDE.
27. Встановіть розширення Go у Visual Studio Code, створіть проєкт, напишіть програму, яка перевіряє, чи є число парним, і запустіть її через вбудований термінал.
28. Напишіть програму, яка виводить числа від 1 до 10, використовуючи цикл **for**.
29. Перепишіть базовий приклад програми, додавши до кожного рядка коментар із поясненням його функції.
30. Створіть програму, яка вітає користувача та запитує його вік. Виведіть повідомлення з розрахунком, скільки років йому буде через 10 років.

Огляд тестових завдань

Закриті запитання з одним варіантом відповіді

Що означає директива `package main` у програмі Go?

- a) Визначає імпорт бібліотеки
- b) Вказує основний пакет програми
- c) Оголошує змінну
- d) Вказує тип даних

Правильна відповідь: b).

Закриті запитання з кількома правильними відповідями

Які середовища підтримують розробку на Go?

- a) Visual Studio Code
- b) Notepad++
- c) LiteIDE
- d) Eclipse

Правильна відповідь: a), b), c).

Завдання на відповідність

Встановіть відповідність між командою та її функцією:

- a) `go build` → 1. Компіляція програми
- b) `go run` → 2. Виконання програми
- c) `fmt.Println` → 3. Виведення тексту

d) package main → 4. Визначення основного пакета

Правильна відповідь:

a) → 1.

b) → 2.

c) → 3.

d) → 4.

Завдання на доповнення коду

Доповніть програму для виведення "Hello, Go!":

```
_____ main
import "fmt"
func main() {
    fmt._____ ("Hello, Go!")
}
```

Правильна відповідь: ...package..., ...Println...

Завдання з вибором істинності

Команда go build створює виконуваний файл програми.

a) Так

b) Ні

Правильна відповідь: a).

Відкриті запитання

Поясніть, чому команда go run не створює виконуваний файл.

Правильна відповідь: Команда go run лише виконує програму, не зберігаючи її як виконуваний файл.

Завдання з кодом

Яка помилка в програмі?

```
package main
import "fmt"
func main {
    fmt.Println("Hello, Go!")
}
```

a) Відсутній імпорт пакета

b) Немає дужок у визначенні функції

- c) Неправильний синтаксис функції Println
- d) Усе правильно

Правильна відповідь: b).

Завдання на завершення коду

Завершіть код, щоб вивести:

The answer is 42.

Правильна відповідь: `fmt.Println("The answer is", 42)`

ПЕРЕЛІК ПОСИЛАНЬ

1. The Go Programming Language.htm [Електронний ресурс]. - Режим доступу: <https://go.dev>. - Назва з екрана.
2. All releases - The Go Programming Language.htm [Електронний ресурс]. - Режим доступу: <https://go.dev/dl>. - Назва з екрана.
3. IDE LiteIDE.htm [Електронний ресурс]. - Режим доступу: <http://liteide.org/en>. - Назва з екрана.
4. Liteide - Browse Files at SourceForge.net.htm [Електронний ресурс]. - Режим доступу: <https://sourceforge.net/projects/liteide/files>. - Назва з екрана.
5. Visual Studio Code - Code Editing. Redefined.htm [Електронний ресурс]. - Режим доступу: <https://code.visualstudio.com>. - Назва з екрана.
6. Standard library - Go Packages.htm [Електронний ресурс]. - Режим доступу: <https://golang.org/pkg>. - Назва з екрана.
7. Fmt package - fmt - Go Packages.htm [Електронний ресурс]. - Режим доступу: <https://golang.org/pkg/fmt>. - Назва з екрана.
8. Net package - net - Go Packages.htm [Електронний ресурс]. - Режим доступу: <https://golang.org/pkg/net>. - Назва з екрана.
9. Http package - net_http - Go Packages.htm [Електронний ресурс]. - Режим доступу: <https://golang.org/pkg/net/http>. - Назва з екрана.
10. Sql package - database_sql - Go Packages [Електронний ресурс]. - Режим доступу: <https://golang.org/pkg/net/http.htm>. - Назва з екрана.
11. Go Wiki_ SQL Database Drivers - The Go Programming Language.htm [Електронний ресурс]. - Режим доступу: <https://go.dev/wiki/SQLDrivers>. - Назва з екрана.
12. GitHub - go-sql-driver_mysql_ Go MySQL Driver is a MySQL driver for Go's (golang) database_sql package.htm [Електронний ресурс]. - Режим доступу: <https://github.com/go-sql-driver/mysql>. - Назва з екрана.
13. GitHub - lib_pq_ Pure Go Postgres driver for database_sql.htm [Електронний ресурс]. - Режим доступу: <https://github.com/lib/pq>. - Назва з екрана.
14. Pq package - github.com_lib_pq - Go Packages.htm [Електронний ресурс]. - Режим доступу: <https://godoc.org/github.com/lib/pq>. - Назва з екрана.
15. GitHub - mattn_go-sqlite3_ sqlite3 driver for go using database_sql.htm [Електронний ресурс]. - Режим доступу: <https://github.com/mattn/go-sqlite3>. - Назва з екрана.
16. Mgo - Rich MongoDB driver for Go.htm [Електронний ресурс]. - Режим доступу: <https://labix.org/mgo>. - Назва з екрана.

17. Gorilla, the golang web toolkit.htm [Електронний ресурс]. - Режим доступу: <http://www.gorillatoolkit.org>. - Назва з екрана.

СПИСОК РЕКОМЕНДОВАНОЇ ЛІТЕРАТУРИ І ЕЛЕКТРОННИХ РЕСУРСІВ

1. Jay McGavren. Head First Go. «O'Reilly Media», 2022 – 556 с. ISBN 978-1491969557.
2. Head First HTML and CSS: A Learner's Guide to Creating Standards-Based Web Pages. O'Reilly Media) 2019. – 762 p. ISBN 978-0596159900.
3. Head First PHP & MySQL. O'Reilly Media) 2018. – 812 p. ISBN 978-0596006303.
4. Програмування вебзастосунків мовою Go [електронний ресурс] // Укладач.: Недашківський О. Л.. - Київ.: КПІ ім. Ігоря Сікорського. - 2024.
5. Head First. Програмування на JavaScript. Фабула. 2022. – 672 p. ISBN 978-617-522-047-4.